



CMP450: Object-Oriented Software Engineering

Spring 2026

IntelliRide: Heuristic-Based Ride-Sharing & Carpool Matching System

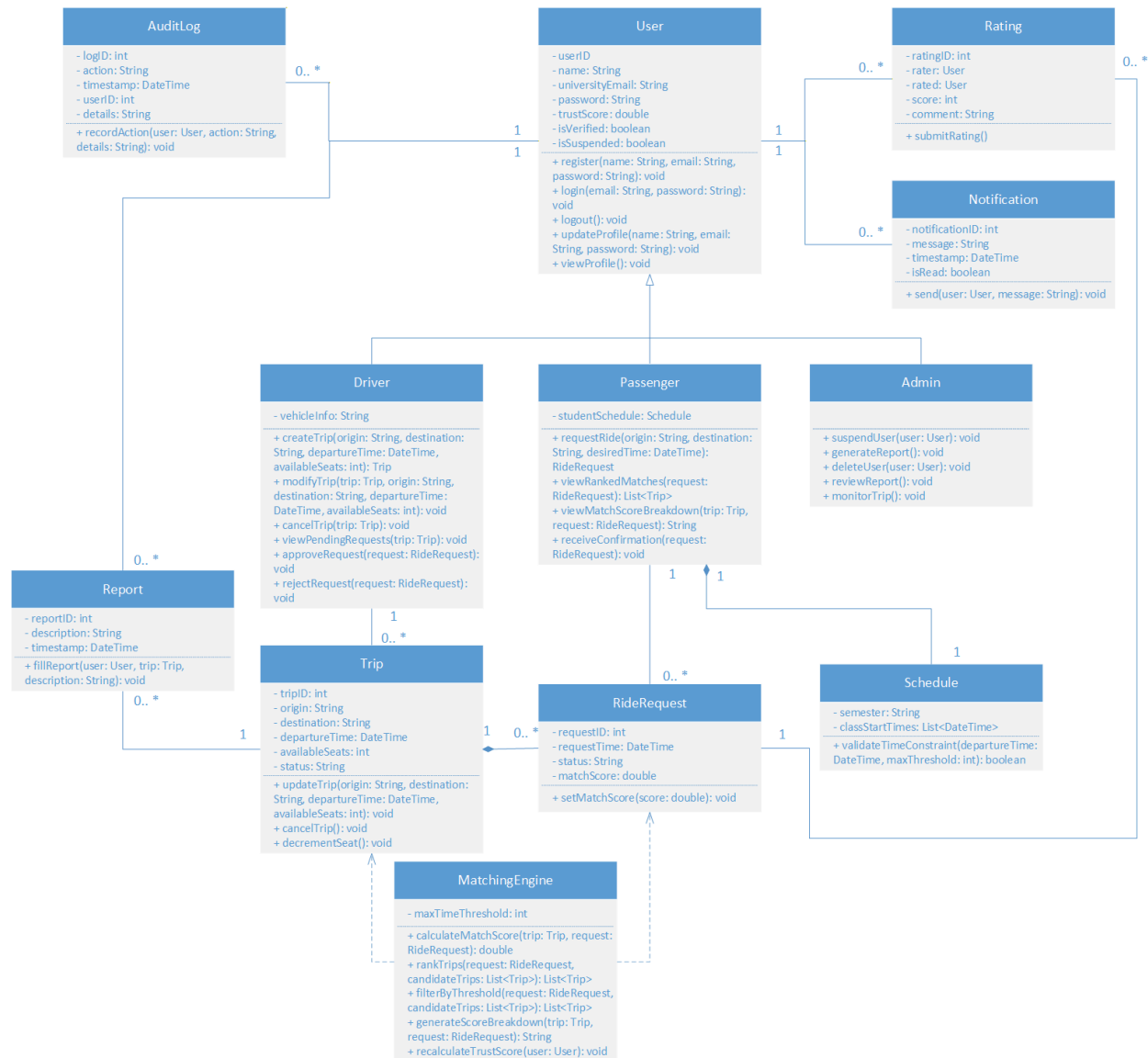
Submitted by:

Name	ID
Mohamed Alawadhi	b00094286
Humaid Almheiri	b00092339
Shaikh Usman	b00092197

Instructor: Dr. Bilal Al-Ahmad

Submission Date: 5th April 2026

1. UML Class Diagram [Structural perspective]



The class diagram for the IntelliRide system represents the structural foundation of the system and captures the main entities, their responsibilities, and their relationships. At its core, the **User** superclass defines shared attributes and behaviors for the **Driver**, **Passenger**, and **Admin** classes, promoting code reuse and consistency. This design allows the users to perform common operations such as registration, authentication, and profile management while also extending functionality through their respective subclasses.

The Driver class models the drivers or vehicle owners who can create, modify, manage, and cancel multiple Trip instances. Each driver can also view pending ride requests, and are able to approve or reject these requests.

The Passenger class represents the typical passengers or the students wanting to go to a specific area. Each passenger can request a ride, view the best matches, view the match score breakdown, and receive confirmation of a ride. The Passenger-to-Schedule relationship is modeled as a composition, as each passenger must have their own student schedule to which the timing of their trip must not conflict with. A passenger can have multiple RideRequest instances. Additionally, each passenger can request a ride (specifying time, destination, and pickup location), view the best matches, view the match score breakdown, and receive confirmation of a ride.

The Admin class serves as the administrator of the system. They can suspend users, delete users, generate statistical reports, review user reports, and monitor trips.

The Trip class models the rides created by drivers. Each trip can be created and updated by exactly one driver, may receive multiple RideRequest instances (composition), has a seat capacity, and can even cancel trips. This class also has a dependency relationship with the MatchingEngine class.

The RideRequest class represents a passenger's request to join a specific trip. Each ride request is created by exactly one passenger and is associated with exactly one trip. It can set the match score between request and trip. This class also has a dependency relationship with the MatchingEngine class.

The MatchingEngine class is responsible for calculating matches based on a heuristic-based matching algorithm. It can calculate the match scores based on the trip and request, rank trips, filter trips exceeding maximum time threshold, provide score breakdowns for transparency, and recalculate trust scores.

The Schedule class contains a student's university class schedule, and it can validate time constraints depending on the ride a student wants to take.

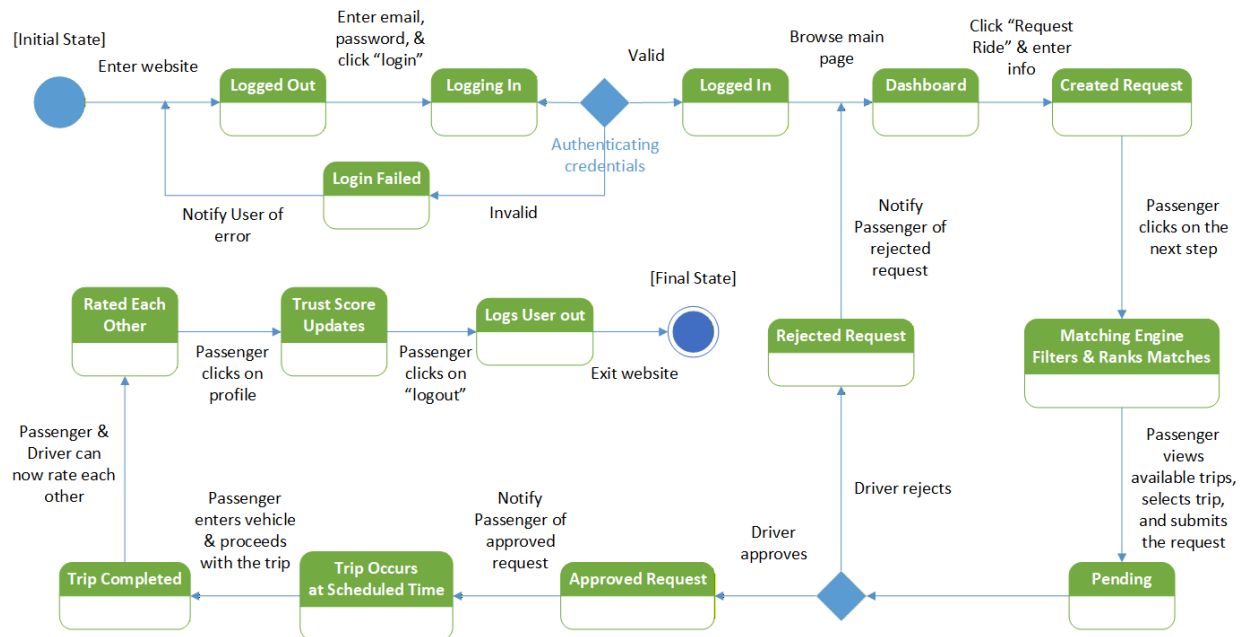
The Report class allows users to report misconduct during or after ride completion. A user can fill multiple reports. The admins can review these reports.

The AuditLog class keeps track of user actions such as login attempts, trip creation, ride approvals, and account suspensions.

The Notification class sends notification messages to the user in case of a request approval/rejection, pending rides, account suspension, and login failures.

The Rating class supports the trust and feedback loop of the system. Each user can submit ratings, can be received by one user, and updates the recipient's trust score dynamically.

2. UML State Diagram [Behavioral perspective]



Once the user enters the website, the system will begin in the “Logged Out” state, where the login screen is displayed. The user would enter their email address and password to login. Upon successful authentication, the user goes to the “Logged In” state. Here, the system determines the user’s role (either Driver, Passenger, or Admin) and loads the corresponding dashboard layout. Passengers can create a ride request by entering their origin, destination, and desired time. Once it has been created, the passenger clicks on the next step where the Matching Engine filters and ranks matches based on the passenger’s request criteria. The passenger views available trips based on the request, selects the most suitable trip, and submits the final request, thus going to the “Pending” state.

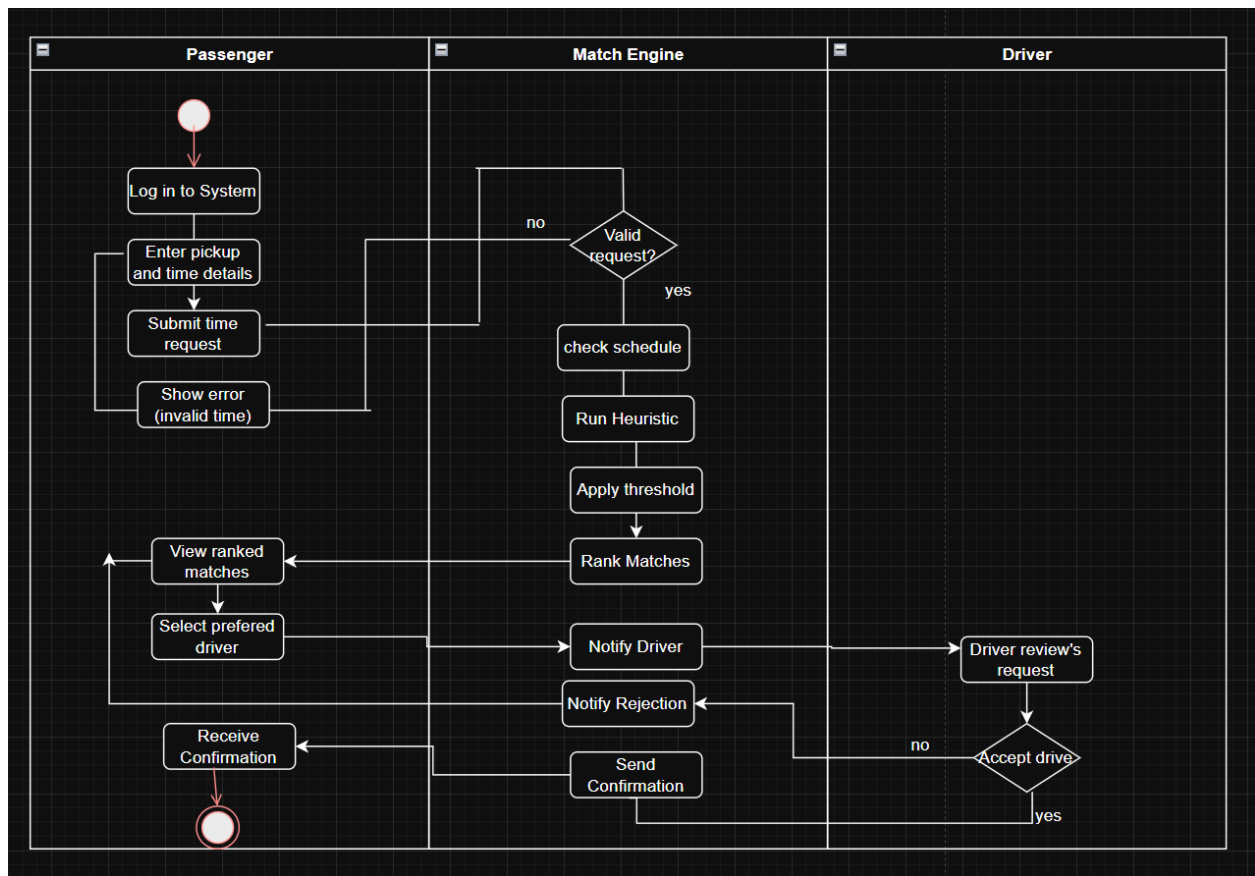
Now that the request is pending, a driver can view the sent request and either approves/rejects the request. In this instance, we shall go with the driver approving the request, to which the system enters its “Approved Request” state. Since the request has been approved by the driver, a notification message is sent to the passenger of the approved request.

The “Trip Occurs at Scheduled Time” state represents the start of the trip and being present at the origin of the passenger. The passenger then enters the vehicle and proceeds with the trip. After a while, the trip is completed, where the system enters the “Trip Completed” state. Both the passenger and driver can rate each other, now that the trip is completed. Once rated, the system

enters the “Rated Each Other” state. Then, the passenger clicks on their profile to view any trust score updates. The passenger then decides to logout and exit the website.

3. UML Activity Diagram [Behavioral perspective] (ONLY one use case)

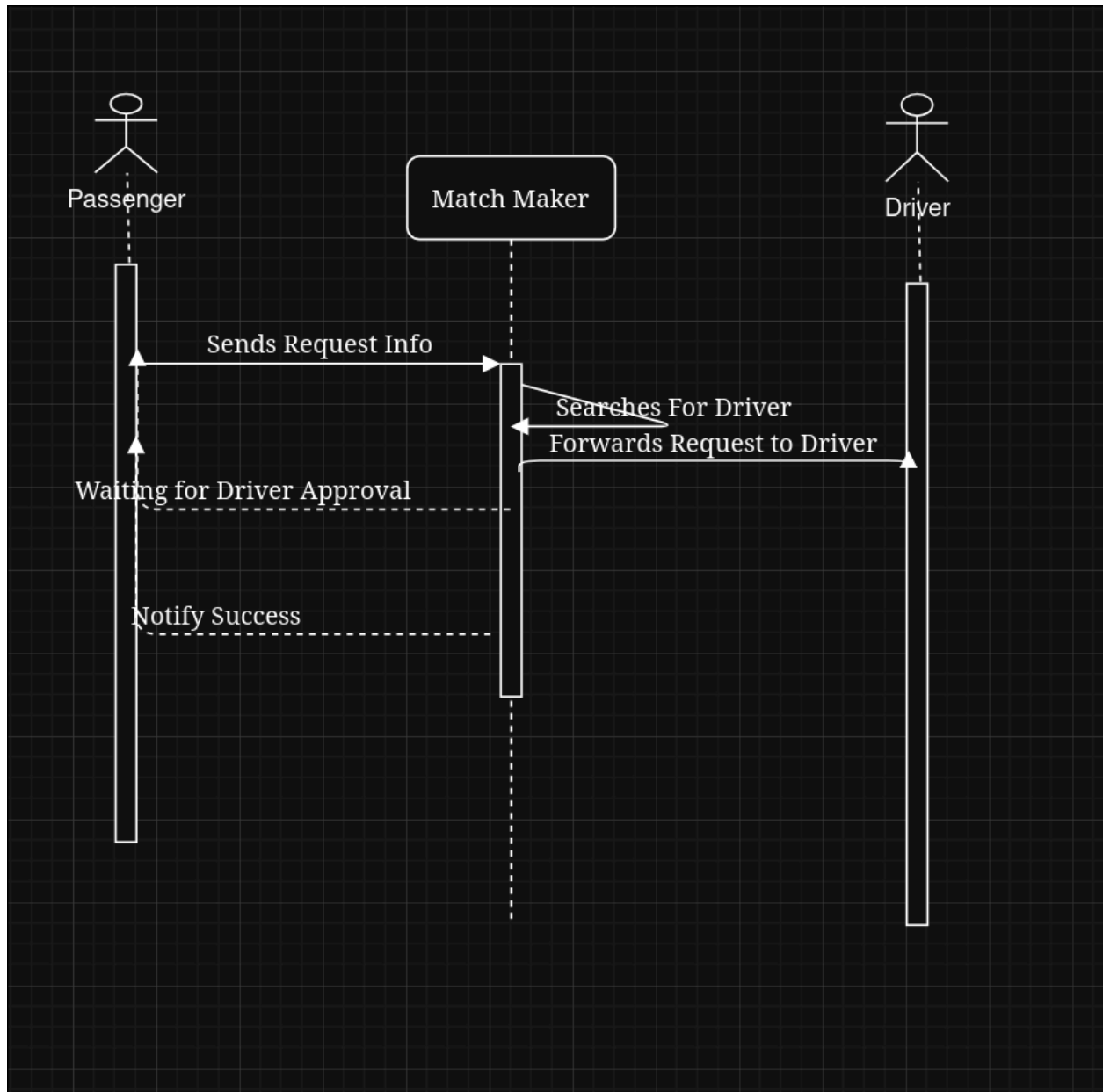
Passenger Requests Ride



Passenger logs into the system and requests a ride by inputting their pick up details, the request is submitted and checked with the match engine to see if the request is valid (i.e. time, location, etc.). If invalid, passenger inputs once again. If valid, engine checks system schedule for space and runs heuristics with available drivers and their locations, applies threshold to find compatible drivers, and ranks the matches which are then reflected to the passenger to pick from. Driver selected is then notified and chooses to either accept or reject. If rejected, the passenger is

notified and selects another driver. If accepted, the passenger receives confirmation and the activity ends.

4. UML Sequence Diagram [Behavioral perspective] (ONLY one use case)



UML Behavioral diagram to represent the process of a passenger finding a driver